



logo_skynet_dark.png

Gruppo SkyNet
skynetunigroup@gmail.com

Analisi dei Requisiti

Versione	<i>Inserire Versione (es. 1.0)</i>
Uso	<i>Esterno</i>
Destinatari	<i>Prof. Tullio Vardanega, Prof. Riccardo Cardin</i>
Redattori	<i>Nome Cognome 1, Nome Cognome 2</i>
Verifica	<i>Nome Cognome 1, Nome Cognome 2</i>
Approvazione	<i>Nome Cognome</i>

[LINK DI SUPORTO PER LO SVILUPPO DA ELIMINARE NEL DOCUMENTO EFFETTIVO]

Slide per l'analisi: <https://www.math.unipd.it/~tullio/IS-1/2025/Dispense/T05.pdf>

Esempi:

- [GammardX](#)
- [Torchlight](#)
- [Merl](#)

Versioni del documento

Ver.	Data	Descrizione	Autore	Verificatore
0.3.0	15/04/2026	Inizio scrittura della sezione “Requisiti”	Sara Ristovic	
0.2.0	12/04/2026	Scrittura della sezione 2	Dario Pirrone	
0.1.0	08/04/2026	Creazione documento e scrittura della sezione 1	Dario Pirrone	

Indice

1 Introduzione

1.1 Scopo del documento

Il presente documento, denominato **Analisi dei Requisiti**, ha l'obiettivo di esporre in modo dettagliato e non ambiguo i requisiti funzionali, qualitativi e di vincolo individuati per il progetto **Code Guardian**. Tali requisiti sono stati delineati a seguito di un'attenta analisi del capitolato e dei successivi incontri con il proponente. Il documento si avvale dell'uso di diagrammi e descrizioni dei **Casi d'Uso (UC)** per illustrare le interazioni tra gli attori esterni e il sistema, fornendo una base solida per le successive fasi di progettazione, codifica e verifica.

1.2 Scopo del prodotto [?]

1.3 Glossario

Per evitare ambiguità terminologiche e garantire una comprensione univoca del documento, i termini tecnici, gli acronimi e le parole che assumono un significato specifico nel contesto del progetto sono riportati nel documento *Glossario*. Questi termini sono contrassegnati nel presente documento da una "G" a pedice ed è un collegamento ipertestuale che rimanda direttamente alla definizione corrispondente nel *Glossario*. Tali parole vengono contrassegnate solamente alla loro prima occorrenza in questo documento.

1.4 Riferimenti

1.4.1 Riferimenti nominativi

- Norme di Progetto v1.0.0
- Piano di Qualifica v1.0.0
- [Capitolato d'appalto C2 - Code Guardian](#)
- [Regolamento del Progetto Didattico](#)

1.4.2 Riferimenti informativi

- Glossario v1.0.0
- [Dispense Analisi dei Requisiti](#)
- [Dispense Casi d'Uso](#)

2 Descrizione generale

2.1 Obiettivi del prodotto

L'obiettivo principale del progetto è lo sviluppo di un set di agenti intelligenti basati su modelli linguistici di grandi dimensioni (*LLM*), progettati per integrarsi in un'architettura

di orchestrazione già esistente fornita dal proponente *Var Group*. Il sistema, denominato **Code Guardian**, mira ad automatizzare i processi di audit, manutenzione e documentazione dei repository software, riducendo il carico di lavoro manuale dei team di sviluppo e migliorando la qualità complessiva del codice.

Nello specifico, il prodotto si prefigge di:

- **Garantire la coerenza documentale:** assicurando che il README di progetto e la documentazione tecnica (inline e API) siano costantemente allineati all'evoluzione funzionale della piattaforma.
- **Elevare gli standard di sicurezza:** introducendo il paradigma di *policy-as-code*, attraverso il quale gli agenti verificano non solo le vulnerabilità generiche OWASP, ma anche l'aderenza alle specifiche pratiche di sviluppo sicuro definite dal team.
- **Migliorare la trasparenza verso il business:** automatizzando la redazione di changelog comprensibili al cliente finale, traducendo i progressi tecnici emersi dalle user stories in valore di business tangibile.

2.2 Funzioni del prodotto

Il prodotto offre le sue funzionalità attraverso tre agenti specializzati coordinati dall'orchestratore centrale:

1. Agente di Documentazione e README:

- **Manutenzione README:** creazione e aggiornamento dinamico del file README di progetto in base alle modifiche apportate alla piattaforma.
- **Gestione documentazione tecnica:** generazione e aggiornamento di documentazione inline (*docstring*, *JSDoc*) e delle *API docs* per riflettere lo stato attuale del codice.

2. Agente di Security Review (Policy-as-Code):

- **Analisi vulnerabilità:** utilizzo delle API di *Claude Code* per l'identificazione di vulnerabilità standard *OWASP*.
- **Verifica policy interne:** controllo dell'adozione di pratiche di sviluppo sicuro specifiche del team, utilizzando i file di configurazione *CLAUDE.md* presenti nel repository come contesto operativo.

3. Agente di Changelog da User Stories:

- **Analisi Sprint:** lettura e filtraggio delle *user stories* completate durante uno specifico sprint di sviluppo.
- **Traduzione Business-Oriented:** conversione del linguaggio tecnico delle task in un linguaggio chiaro e comprensibile per il cliente, producendo *changelog* sia tecnici che di business.

2.3 Caratteristiche degli utenti

Il sistema è progettato per inserirsi in un flusso di lavoro di sviluppo software moderno, interagendo con diverse figure professionali che operano all'interno di un team IT o nel rapporto con il cliente finale.

L'utente principale è rappresentato dallo **sviluppatore**, il quale interagisce con il sistema indirettamente attraverso il repository Git. Egli beneficia dell'automatizzazione della documentazione tecnica e del feedback immediato sulla sicurezza, che gli permette di correggere eventuali vulnerabilità o difformità rispetto alle policy aziendali in tempo reale, senza attendere revisioni manuali.

Una seconda figura chiave è il **Team Lead** o lo **Security Auditor**. Questi utenti utilizzano il sistema come strumento di monitoraggio e governance: definiscono le regole di "sviluppo sicuro" all'interno dei file CLAUDE.md e delegano all'agente di sicurezza il compito di verificarne l'adozione costante su tutto il codice prodotto.

Infine, il sistema si rivolge agli **Stakeholder di business** (come Project Manager o il cliente finale). Questi soggetti non necessitano di competenze tecniche approfondite, ma hanno bisogno di visibilità sull'andamento del progetto. Per loro, l'agente di changelog funge da "traduttore", trasformando il gergo tecnico delle user stories in report di valore che sintetizzano i progressi funzionali raggiunti al termine di ogni sprint.

2.4 Piattaforma di esecuzione [?]

Il prodotto è concepito come un ecosistema di agenti integrati in un'infrastruttura già esistente, operante principalmente in ambiente **cloud AWS**. Essendo il sistema basato su interfacce di programmazione e automazione, l'esecuzione è garantita attraverso i principali ambienti di runtime come **Node.js** e **Python**. L'interazione con il codice sorgente avviene tramite la piattaforma **GitHub**, sfruttando le **GitHub Actions** come principale motore di esecuzione per l'analisi e l'aggiornamento automatico dei repository.

3 Casi d'uso

3.1 Introduzione

I Casi d'Uso sono costituiti da un diagramma UMLG e da una descrizione testuale che ne chiarisce il comportamento e gli obiettivi, al fine di definire in modo preciso le funzionalità che il prodotto deve offrire.

3.2 Attori

3.3 Elenco dei casi d'uso

Per ogni agente preso in esame, verrà sviluppata una sezione dell'Analisi dei Requisiti dedicata, così da migliorare l'organizzazione del documento.

4 Agente 1: Documentazione e README

4.1 Introduzione all'agente

L'Agente di Documentazione e README rappresenta un pilastro fondamentale per la qualità del lavoro quotidiano degli sviluppatori e per la stabilità del progetto Code Guardian. Sappiamo bene come, nei cicli di sviluppo veloci, la documentazione sia spesso la prima a perdere il passo rispetto al codice, diventando presto superata. Questo agente nasce proprio per risolvere tale problema: il suo compito è automatizzare tutto il processo di aggiornamento. In questo modo, garantiamo che il file README.md e l'intera documentazione tecnica, sia quella interna al codice che quella dedicata alle API, siano sempre uno specchio fedele e puntuale dell'evoluzione del progetto, in qualsiasi momento.

4.2 Obiettivi e Funzionalità

L'agente si focalizza su tre aree critiche:

1. **Manutenzione del README:** Automazione della creazione e dell'aggiornamento del file principale di presentazione del repository, adattandolo dinamicamente alle evoluzioni della piattaforma.
2. **Gestione della documentazione inline:** Analisi semantica del codice per la generazione di *docstring* e *JSDoc*, assicurando che ogni modulo sia correttamente commentato.
3. **Documentazione delle API:** Estrazione automatizzata degli endpoint per la creazione di guide tecniche per i consumatori del servizio.

4.3 Casi d'Uso

Di seguito i casi d'uso dell'agente 1.

4.3.1 UC 1: Gestione del README di Progetto

Questo caso d'uso riguarda il mantenimento del file README.md principale per riflettere lo stato attuale del software.

- **Attore Primario:** Sviluppatore.
- **Attori Secondari:** Orchestratore, Repository Git.
- **Trigger:** Lo Sviluppatore effettua un push di modifiche strutturali o richiede l'aggiornamento tramite l'Orchestratore.
- **Precondizioni:** L'Agente 1 ha i permessi di lettura/scrittura sul repository; esiste un file README.md.
- **Scenario Principale:**
 1. L'Orchestratore notifica l'Agente 1 della necessità di aggiornamento.

2. L'Agente analizza file di configurazione (es. `package.json`) e struttura cartelle.
 3. L'Agente identifica nuove dipendenze o cambi di versione.
 4. L'Agente rigenera le sezioni interessate (es. guida all'installazione).
 5. L'Agente propone la modifica tramite Pull Request.
- **Post-condizioni:** Il file `README.md` è aggiornato e coerente con l'ultima versione del codice.

4.3.2 UC 2: Generazione Documentazione Inline (Docstring/JSDoc)

L'agente interviene nel codice sorgente per garantire che ogni componente sia documentato correttamente.

- **Attore Primario:** Sviluppatore.
- **Attori Secondari:** Orchestratore.
- **Trigger:** Rilevamento di una funzione o classe priva di documentazione o con commenti obsoleti.
- **Scenario Principale:**
 1. L'Agente interpreta la logica della funzione, i parametri di input e il valore di ritorno.
 2. L'Agente genera il blocco di commento standardizzato (JSDoc per JS/TS o Docstring per Python).
 3. L'Agente inserisce il commento sopra la firma della funzione.
- **Sottocasi:**
 - **UC 2.1:** Correzione di documentazione esistente non più coerente (es. parametri cambiati).
 - **UC 2.2:** Segnalazione di logiche troppo complesse per la documentazione automatica.

4.3.3 UC 3: Generazione Documentazione API

L'agente estrae le definizioni degli endpoint per creare una guida tecnica per i consumatori delle API.

- **Attore Primario:** Sviluppatore.
- **Trigger:** Richiesta esplicita di generazione prima di un rilascio o rilevamento nuove rotte.
- **Scenario Principale:**
 1. L'Agente scansiona i file dei controller e i file delle rotte.

2. Estrazione di: metodo HTTP, URL, header, payload richiesto e risposte possibili.
 3. Formattazione delle informazioni nel file `API.md` in formato Markdown.
 4. Salvataggio del documento nella cartella di documentazione predefinita.
- **Post-condizioni:** Disponibilità di un documento tecnico (`API.md`) aggiornato per gli stakeholder.

4.4 Requisiti Funzionali

I requisiti funzionali per l'Agente 1 sono definiti per rispondere alle esigenze di coerenza documentale espresse dal proponente:

- **RF.1.1 (Aggiornamento README):** Il sistema deve identificare le modifiche strutturali nel repository (es. aggiunta di moduli) e aggiornare le sezioni relative all'installazione nel `README.md`.
- **RF.1.2 (Standardizzazione Inline):** Il sistema deve analizzare le funzioni prive di documentazione e generare commenti standardizzati che descrivano parametri di input, output e comportamenti attesi.
- **RF.1.3 (Workflow di validazione):** Tutte le modifiche documentali proposte dall'agente devono essere sottomesse tramite *Pull Request*, subordinando l'applicazione delle stesse a una revisione umana obbligatoria per garantire l'accuratezza del contenuto.
- **RF.1.4 (Documentazione API):** Il sistema deve scandire i *controller* dell'applicazione per produrre o aggiornare una guida Markdown (es. `API.md`) che elenchi metodi HTTP, rotte e payload richiesti.

5 Agente 4: Changelog da User Stories

5.1 Introduzione all'agente

L'Agente di Changelog da User Stories funge da mediatore semantico tra il progresso puramente tecnico del team di sviluppo e le necessità informative degli stakeholder di business. In un contesto Agile, comprendere l'impatto reale di uno Sprint può essere complesso per figure non tecniche se i resoconti si limitano a liste di commit o ticket criptici. Questo agente automatizza la traduzione delle attività concluse in report chiari, garantendo che ogni aggiornamento del software sia accompagnato da una narrazione del valore di business generato, senza però tralasciare il dettaglio tecnico necessario alla manutenzione interna.

5.2 Obiettivi e Funzionalità

L'agente si concentra sulla sintesi e trasformazione del progresso di progetto attraverso tre direttrici:

1. **Recupero e Analisi dei Dati:** Integrazione con i sistemi di task management (Jira, GitHub Issues) per l'estrazione automatizzata dei task completati.
2. **Sintesi Semantica Multitarget:** Generazione di report differenziati; uno tecnico (Changelog Tecnico) per il team di sviluppo e uno di valore (Changelog di Business) per i clienti e il management.
3. **Validazione e Qualità del Contesto:** Monitoraggio della qualità informativa dei task per prevenire la generazione di report ambigui o privi di valore.

5.3 Casi d'Uso

Di seguito i casi d'uso dell'agente 4.

5.3.1 UC 7: Recupero e Filtraggio User Stories

Questo caso d'uso riguarda l'estrazione delle attività completate dai sistemi esterni per avviare il processo di analisi.

- **Attore Primario:** Sviluppatore / Project Manager.
- **Attori Secondari:** Orchestratore, Sistema di Task Management (GitHub/Jira).
- **Trigger:** Lo Sviluppatore richiede la generazione del report a conclusione di uno Sprint.
- **Precondizioni:** L'Agente ha i permessi di lettura sulle API del sistema di tracking; le task sono in stato "Done".
- **Scenario Principale:**
 1. L'Agente interroga il sistema tramite API filtrando per Sprint ID e stato.
 2. L'Agente valida la presenza di titoli e descrizioni nelle User Stories recuperate.
 3. L'Agente organizza i dati separandoli per tipologia (bugfix, feature, task tecnici).
- **Sottocasi:**
 - **UC 7.1:** Rilevamento di task con metadati insufficienti e segnalazione all'utente.
 - **UC 7.2:** Gestione delle ambiguità se l'AI rileva discrepanze tra titolo e contenuto tecnico.

5.3.2 UC 8: Generazione Changelog Tecnico (Dev-to-Dev)

L'agente produce un documento rivolto ai programmatori che riassume i cambiamenti a basso livello.

- **Attore Primario:** Sviluppatore.

- **Attori Secondari:** Orchestratore.
- **Scenario Principale:**
 1. L'Agente aggrega le User Stories per area di impatto tecnico.
 2. Estrazione dei requisiti tecnici soddisfatti e dei riferimenti (ID task).
 3. Formattazione del testo in linguaggio tecnico asciutto in formato Markdown (CHANGELOG.md).

5.3.3 UC 9: Generazione Changelog di Business (Dev-to-Client)

L'agente esegue una traduzione semantica per trasformare il gergo tecnico in benefici tangibili.

- **Attore Primario:** Stakeholder / Cliente.
- **Attori Secondari:** Orchestratore, Modello AI (LLM).
- **Precondizioni:** Disponibilità dei dati dello Sprint; configurazione dell'audience (UC 12.5).
- **Scenario Principale:**
 1. L'Agente interpreta il valore delle User Stories per l'utente finale tramite LLM.
 2. L'AI converte termini tecnici in benefici (es. "Migliorata velocità" invece di "Ottimizzazione indici").
 3. Filtro dei dettagli tecnici irrilevanti per lo stakeholder non tecnico.
 4. Generazione di una bozza di report (PDF o testo) pronta per la revisione.

5.3.4 UC 13.5: Validazione Formato Output AI

Questo caso d'uso gestisce le situazioni in cui il modello linguistico produce un output non conforme agli standard tecnici richiesti.

- **Attore Primario:** Sviluppatore.
- **Attori Secondari:** Modello AI (LLM).
- **Trigger:** L'output generato dall'AI presenta errori di formattazione (es. Markdown corrotto o JSON non valido).
- **Scenario Principale:**
 1. L'Agente riceve la risposta dall'LLM e ne analizza la struttura sintattica.
 2. Se vengono rilevati errori, l'Agente richiede automaticamente una correzione al modello (Self-Correction).
 3. In caso di persistenza dell'errore, il sistema notifica lo Sviluppatore dell'impossibilità di generare il report nel formato richiesto.

5.3.5 UC 14.4: Esportazione Report Multiformato

L'agente permette la distribuzione dei report approvati in diversi formati a seconda del destinatario.

- **Attore Primario:** Project Manager / Sviluppatore.
- **Scenario Principale:**
 1. L'utente seleziona il report approvato e sceglie il formato (PDF, Markdown, JSON).
 2. Il sistema applica i template grafici o strutturali richiesti.
 3. Il sistema genera il file per il download o il salvataggio automatico nel repository.

5.4 Requisiti Funzionali

I requisiti funzionali per l'Agente 4 definiscono le capacità di analisi e trasformazione dei dati richiesti:

- **RF.4.1 (Integrazione Sistemi Task):** Il sistema deve interfacciarsi con le API dei sistemi di Task Management esterni per il recupero dei metadati delle User Stories.
- **RF.4.2 (Generazione Multitarget):** Il sistema deve poter generare simultaneamente un report tecnico (orientato allo sviluppo) e un report di business (orientato al cliente).
- **RF.4.3 (Traduzione Semantica):** Il sistema deve utilizzare modelli di linguaggio (LLM) per mappare le attività tecniche a benefici funzionali descrittivi.
- **RF.4.4 (Validazione Dati):** Il sistema deve segnalare all'utente i task completati che presentano descrizioni ambigue o insufficienti per una corretta analisi automatizzata.

6 Note Operative

L'Agente 1 non è un software che richiede un intervento manuale costante. Esso opera in background: ogni volta che viene inviato nuovo codice al repository, l'agente riceve una notifica automatica. Invece di rielaborare l'intero progetto, esamina solo le parti che sono state effettivamente modificate. Questo permette di mantenere la documentazione aggiornata senza rallentare il flusso di lavoro degli sviluppatori, che ricevono solo le proposte di modifica finali pronte per essere revisionate.

6.0.1 Agente 3:

UC 4: Esecuzione Scansione Vulnerabilità (OWASP)

L'agente utilizza le API di Claude Code per identificare vulnerabilità standard nel codice sorgente.

- Attore Primario: Sviluppatore.
- Attori Secondari: Orchestratore, API Claude Code.
- Trigger: Lo Sviluppatore richiede una scansione di sicurezza o l'Orchestratore avvia il processo dopo un commit nel repository.
- Precondizioni:
 - L'Agente 3 è correttamente configurato con le API Key di Claude Code.
 - Il codice sorgente è accessibile nel repository.
- Scenario Principale:
 1. L'Orchestratore invia i file sorgente (o il diff dell'ultimo commit) all'Agente 3.
 2. L'Agente 3 interroga le API di Claude Code inviando il codice per l'analisi.
 3. L'AI analizza il codice cercando pattern riconducibili alle vulnerabilità della Top 10 OWASP (es. SQL Injection, XSS, Hardcoded secrets).
 4. L'Agente 3 riceve l'esito dell'analisi.
 5. L'Agente 3 formatta i risultati evidenziando la gravità (High, Medium, Low) di ogni falla trovata.
- Post-condizioni: Viene generato un report tecnico delle vulnerabilità rilevate.
- Sottocasi:
 - UC 4.1: Scansione specifica su un singolo file modificato. (include)
 - UC 4.2: Scansione completa dell'intero repository (Full Audit). (include)

UC 5: Verifica Conformità "Policy-as-Code" (CLAUDE.md)

L'agente verifica che il codice rispetti le regole specifiche definite dal team nel file di configurazione.

- Attore Primario: Sviluppatore.
- Attori Secondari: Orchestratore, File di configurazione CLAUDE.md.
- Trigger: Avvio della revisione di sicurezza (manuale o automatica).

- Precondizioni: Nel repository è presente un file denominato CLAUDE.md contenente le linee guida di sviluppo sicuro del team.
- Scenario Principale:
 1. L'Agente 3 legge il contenuto del file CLAUDE.md per estrarre il contesto e le regole (es. "usa sempre questa libreria per il logging", "non usare variabili globali").
 2. L'Agente analizza il codice sorgente confrontandolo con le direttive estratte.
 3. L'Agente identifica eventuali deviazioni dalle pratiche concordate dal team.
 4. L'Agente genera un elenco di violazioni della policy interna.
- Post-condizioni: Viene prodotto un elenco di non-conformità rispetto alle policy di progetto.
- Sottocasi:
 - UC 5.1: Validazione del file CLAUDE.md (verifica che il file di configurazione sia scritto correttamente). (extend)

UC 6: Generazione Suggerimenti di Remediation

L'agente fornisce soluzioni pratiche per correggere i problemi di sicurezza o le violazioni di policy trovate.

- Attore Primario: Sviluppatore.
- Attori Secondari: API Claude Code.
- Trigger: Conclusione con esito positivo (rilevamento falle) degli UC 4 o UC 5.
- Precondizioni: Sono state identificate vulnerabilità o violazioni di policy.
- Scenario Principale:
 1. Per ogni vulnerabilità o violazione trovata, l'Agente 3 richiede a Claude Code una possibile correzione.
 2. L'AI genera uno snippet di codice "sicuro" che risolve il problema.
 3. L'Agente presenta allo sviluppatore il confronto tra il codice insicuro e la soluzione proposta.
- Post-condizioni: Lo sviluppatore riceve indicazioni chiare su come correggere il codice per renderlo conforme.

6.0.2 Agente 4:

UC 7: Recupero e Filtraggio User Stories

L'agente deve interfacciarsi con il sistema di gestione dei task per raccogliere il materiale necessario al report.

- Attore Primario: Sviluppatore / Project Manager.
- Attori Secondari: Orchestratore, Sistema di Task Management (es. GitHub Issues API, Jira API).
- Trigger: Lo Sviluppatore richiede la generazione del report a conclusione di uno Sprint.
- Precondizioni: 1. L'Agente ha accesso in lettura alle API del sistema di tracking. 2. Le User Stories sono etichettate con lo Sprint ID corretto e sono nello stato "Done" (Completed).
- Scenario Principale:
 1. L'Orchestratore richiede all'Agente 4 di analizzare lo Sprint corrente (es. "Sprint 5").
 2. L'Agente interroga il sistema di task management filtrando per ID Sprint e stato della storia.
 3. L'Agente valida i dati recuperati (verifica che ci siano descrizioni e titoli validi).
 4. L'Agente organizza internamente le informazioni (separazione tra bugfix, nuove feature e task tecnici).
- Post-condizioni: L'Agente dispone di un set di dati strutturato pronto per la trasformazione.
- Sottocasi:
 - UC 7.1: Segnalazione di User Stories prive di descrizione sufficiente.

UC 8: Generazione Changelog Tecnico

L'agente produce un documento rivolto ai programmatori che riassume i cambiamenti a basso livello.

- Attore Primario: Sviluppatore.
- Attori Secondari: Orchestratore.
- Trigger: Conclusione con successo dell'UC 7.
- Precondizioni: Disponibilità dei dati filtrati dello Sprint.
- Scenario Principale:

1. L'Agente aggrega le User Stories per tipologia tecnica (es. Backend, Frontend, DevOps).
 2. Per ogni storia, estrae i requisiti tecnici soddisfatti e i file modificati (se accessibili).
 3. L'Agente formatta il testo in linguaggio tecnico coerente (es. "Aggiornamento schema DB", "Refactoring API Auth").
 4. Produce un file (solitamente CHANGELOG.md) con link diretti ai task.
- Post-condizioni: È disponibile un changelog dettagliato per l'uso interno del team di sviluppo.

UC 9: Generazione Changelog di Business

L'agente trasforma il gergo tecnico in benefici tangibili per il cliente o l'utente finale.

- Attore Primario: Cliente / Stakeholder.
- Attori Secondari: Orchestratore, Modello AI (LLM).
- Trigger: Conclusione con successo dell'UC 7.
- Precondizioni: Disponibilità dei dati filtrati dello Sprint.
- Scenario Principale:
 1. L'Agente analizza le User Stories e le modifiche apportate.
 2. L'AI esegue una traduzione semantica: converte i termini tecnici in termini di valore (es. "Ottimizzazione indici DB" diventa "Migliorata la velocità di caricamento delle tabelle").
 3. L'Agente rimuove i dettagli tecnici irrilevanti per il cliente (es. bugfix minori di refactoring).
 4. Viene generato un report discorsivo e orientato al business (es. "Novità dello Sprint 5 per l'utente").
- Post-condizioni: Viene prodotto un documento pronto per essere presentato agli stakeholder non tecnici.
- Sottocasi:
 - UC 9.1: Adattamento del tono (formale/informale) in base alla configurazione.

7 Requisiti

I requisiti riportati in questa sezione sono stati derivati dai casi d'uso descritti nel capitolo 3 e dalle indicazioni emerse dagli incontri con il proponente. In particolare, il perimetro del progetto è stato chiarito come sviluppo di agenti specializzati da integrare in un'architettura già esistente, e non come realizzazione di una nuova architettura completa ad agenti.

7.1 Requisiti funzionali

Codice	Priorità	Descrizione	Fonti
RF.1	Obbligatorio	Il sistema deve aggiornare il file README.md del repository in modo coerente con lo stato corrente del codice sorgente.	UC1
RF.2	Obbligatorio	Il sistema deve generare o aggiornare la documentazione inline del codice, come docstring o JSDoc, per elementi sorgente supportati.	UC2
RF.3			
RF.4	Obbligatorio	Il sistema deve eseguire un'analisi di sicurezza del codice alla ricerca di vulnerabilità riconducibili alle categorie OWASP rilevanti.	UC4
RF.4.1	Obbligatorio	Il sistema deve poter eseguire una scansione di sicurezza su un singolo file modificato.	UC4.1
RF.4.2	Obbligatorio	Il sistema deve poter eseguire una scansione di sicurezza sull'intero repository.	UC4.2

7.2 Requisiti qualitativi

Codice	Priorità	Descrizione	Fonti
RQ.1	Obbligatorio	Gli output prodotti dagli agenti devono risultare utili al miglioramento del codice, ad es. tramite segnalazioni, suggerimenti, frammenti di codice o proposte di soluzione.	Verbale esterno
RQ.2	Obbligatorio	Devono essere definite modalità di test e validazione per gli agenti sviluppati.	Verbale esterno

Codice	Priorità	Descrizione	Fonti
RV.1	Obbligatorio	Gli agenti sviluppati devono integrarsi in un'architettura già esistente fornita dal proponente.	Verbale esterno
RV.2	Obbligatorio	Il progetto non deve prevedere l'addestramento di nuovi modelli, ma l'integrazione di modelli o servizi tramite API.	Verbale esterno
RV.3	Obbligatorio	Il sistema deve poter operare sui repository di esempio e sul codice reso disponibile dal proponente per integrazione e test.	Verbale esterno
RV.4	Obbligatorio	Il progetto non prevede lo sviluppo di agenti: orchestratore, per aggiornamento librerie, per sostituzione del codice obsoleto e per mantenimento della copertura dei test, in quanto sono già in sviluppo presso Var Group.	Verbale esterno

7.3 Requisiti di vincolo

7.4 Requisiti di sistema operativo [??]

7.5 (facciamo un servizio web per cui non ci vogliono questi requisiti?)

7.6 Requisiti prestazionali [?]

7.7 Requisiti di sicurezza

Codice	Priorità	Descrizione	Fonti
RS.1	Obbligatorio	Le eventuali modifiche proposte al codice o ai file del repository non devono essere applicate senza validazione umana preventiva.	Verbale esterno
RS.2	Obbligatorio	I risultati dell'analisi di sicurezza devono riportare almeno il tipo di problematica rilevata e una sua priorità o gravità.	UC4

7.8 Tracciamento

7.8.1 Fonte - Requisito

7.8.2 Requisito - Fonte

7.8.3 Riepilogo requisiti